

# Phát Triển Ứng Dụng Đọc Truyện Tranh Hỗ Trợ Đa Ngôn Ngữ Bằng Việc Ứng Dụng AI Tự Động Dịch

*Nghiên cứu và tích hợp các mô hình học sâu (OCR, Inpainting, LLM) và kiến trúc xử lý bất đồng bộ*

**Sinh viên thực hiện:** Nhóm sinh viên thực hiện  
**Đơn vị:** Trường Đại học Công nghệ - ĐHQGHN

**Giảng viên hướng dẫn:** Giảng viên hướng dẫn

# 1. Đặt vấn đề & Mục tiêu đề tài

---

## Đặt vấn đề / Hiện trạng:

- Quy trình dịch thuật truyền thống (**Scanlation**) thủ công tốn thời gian, chi phí lớn.
- Chữ dịch bị **nhúng cứng (hardcoded)** vào điểm ảnh, làm mất thông tin số và khả năng tương tác.

## Mục tiêu cốt lõi:

- Tự động hóa quy trình dịch thuật bằng **Pipeline AI bất đồng bộ**.
- **Tách biệt** lớp ảnh nền sạch (Cleaned Background) và siêu dữ liệu văn bản dịch.
- Tính năng tương tác **Tap-to-Translate** hỗ trợ tra từ vựng theo ngữ cảnh khi đọc.

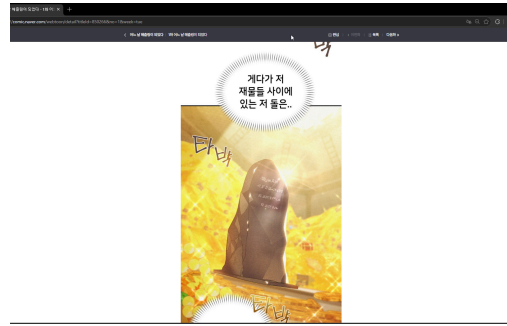
## 2. Định dạng truyện tranh & Thách thức AI

### Đặc điểm định dạng:

- **Manga:** Đọc phải ← trái; chữ dọc; bong bóng thoại elip đứng; từ tượng thanh đè nền.
- **Webtoon:** Cuộn dọc vô hạn; khoảng trống lớn; bong bóng thưa; chữ ngang.



Manga (Page-based)



Webtoon (Scroll)

### Thách thức kỹ thuật:

- **Hình học bong bóng thoại:** Biến dạng đa dạng (elip, tia chớp, mây, không viền).
- **Giao thoa đồ họa:** Chữ đè lên chi tiết nền phức tạp (tóc, bối cảnh).
- **Thứ tự đọc (Reading Order):** Sắp xếp logic ô chữ theo mạch truyện.

### 3. Giải pháp đề xuất & Mô hình AI

---

#### Phân tách khung tranh (Panel)

- Sử dụng **YOLOv12**: trích xuất đặc trưng đa tầng và cơ chế chú ý cải tiến, nhận diện ranh giới ô tranh bất đối xứng rất nhanh và chính xác.

#### Nhận dạng chữ (OCR)

- **Manga**: DBNet (comic-text-detector) định vị bong bóng thoại + Manga0cr nhận diện chữ dọc nghệ thuật.
- **Webtoon**: PaddleOCR-v5 nhận diện ngang đa hướng.

#### Xóa chữ & Phục hồi ảnh nền (Inpainting)

- **Mạng học sâu LaMa**: Sử dụng **Fast Fourier Convolution (FFC)** để có trường thụ cảm toàn cục.
- **Kết quả**: Tự động khôi phục cấu trúc lưới bóng (screentone) và họa tiết nền tự nhiên mà không bị nhòe mờ.

## 4. Dịch thuật ngữ cảnh với Gemini LLM

### Hạn chế dịch thô (Text-only API):

- Không có ngữ cảnh hình ảnh để xác định giới tính/biểu cảm → Sai đại từ xưng hô (“tớ - cậu”, “ta - người”).

### Giải pháp với Gemini 3.1 Flash-Lite:

- **Visual Context-Aware:** Truyền đồng thời văn bản gốc và ảnh **Set-of-Mark (SoM)** (ảnh có vẽ các ô chữ được đánh dấu màu) để LLM đối chiếu ngữ cảnh hình ảnh.
- **Định dạng đầu ra có cấu trúc:** Sử dụng cơ chế JSON Schema ràng buộc chặt chẽ để trả về dữ liệu chuẩn xác.

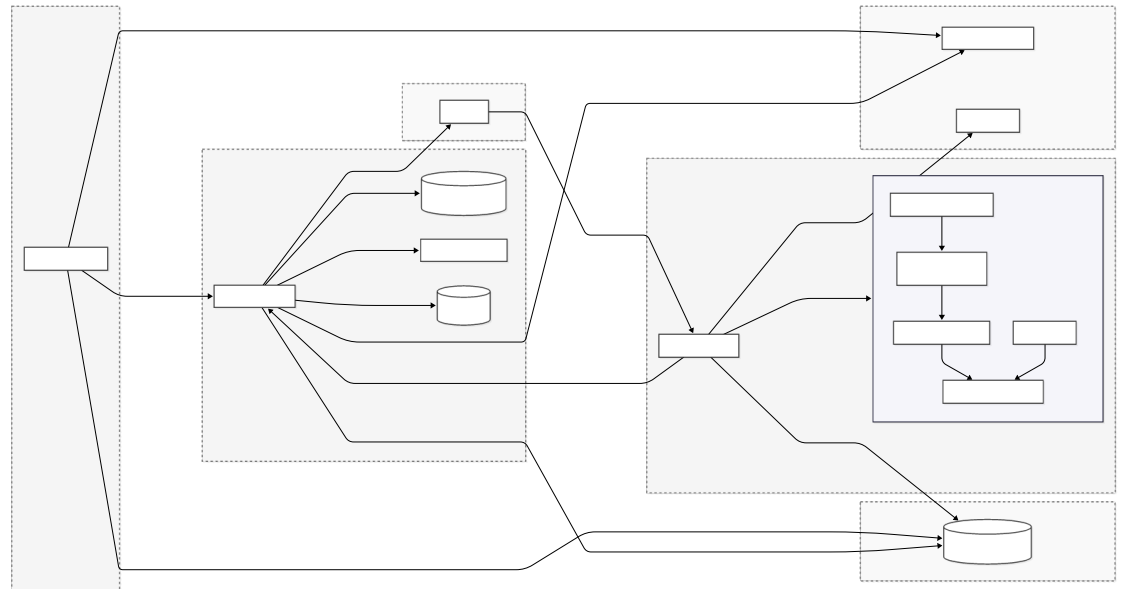
### Đóng gói JSON Output

```
{
  "page_id": "p001",
  "bubbles": [
    {
      "id": 1,
      "full_translation": "Cậu đang làm gì
thê?",
      "chunk_meanings": [...]
    }
  ]
}
```

## 5. Kiến trúc tổng thể hệ thống

### Kiến trúc Hướng dịch vụ Lai (Hybrid SOA):

- **Tầng Giao diện:** Next.js 16  
(Comic Viewer, Tap-to-Translate, SSR/ISR).
- **Dịch vụ Cốt lõi:** Spring Boot 3.5  
(Auth JWT, Content management, ES Sync).
- **Dịch vụ AI:** FastAPI (RabbitMQ, Pipeline OCR/Inpaint, Gemini).
- **Tầng Dữ liệu:** PostgreSQL  
(ACID), Elasticsearch (Fuzzy search), Redis (Cache), MinIO  
(Storage).



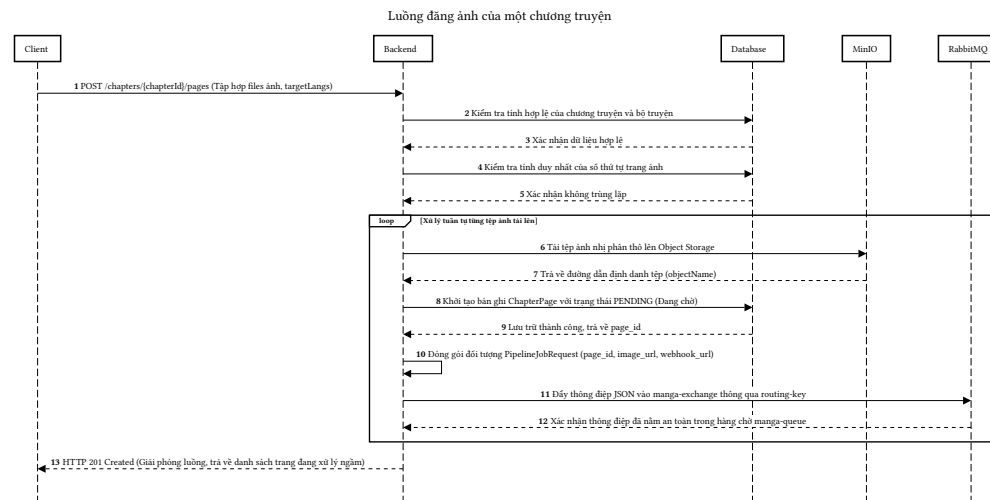
## 6. Xử lý bất đồng bộ qua RabbitMQ

### Nút thắt cổ chai hệ thống:

- Xử lý AI tốn tài nguyên và thời gian.
- HTTP đồng bộ dễ gây **Thread Starvation** và **HTTP Timeout**.

### Cơ chế RabbitMQ bất đồng bộ:

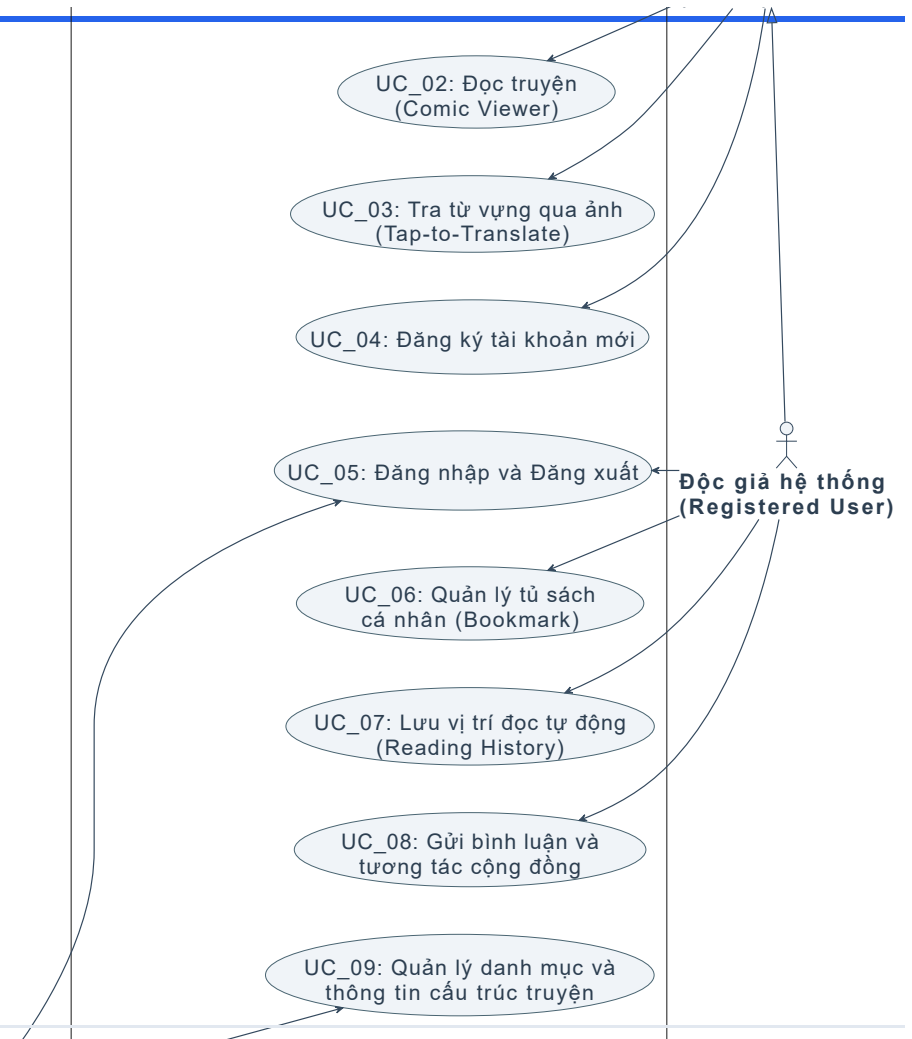
1. **Spring Boot**: Nhận file, lưu MinIO, đẩy Job Payload vào Queue → phản hồi ngay **HTTP 201**.
2. **RabbitMQ Queue**: Hàng đợi lưu đệm, co giãn tải tự động.
3. **FastAPI AI Worker**: Lắng nghe Queue, chạy AI pipeline độc lập.
4. **Callback**: POST Webhook cập nhật kết quả.



## 7. Phân tích yêu cầu chức năng

### Phân nhóm vai trò (Roles):

- **Guest:** Duyệt & tìm kiếm (Elasticsearch), đọc thử, trải nghiệm **Tap-to-Translate**.
- **User:** Quản lý tủ sách, đồng bộ tiến độ đọc, bình luận phân nhánh.
- **Admin:** Quản lý nội dung (truyện/chương), kích hoạt **AI Pipeline**, phân quyền.

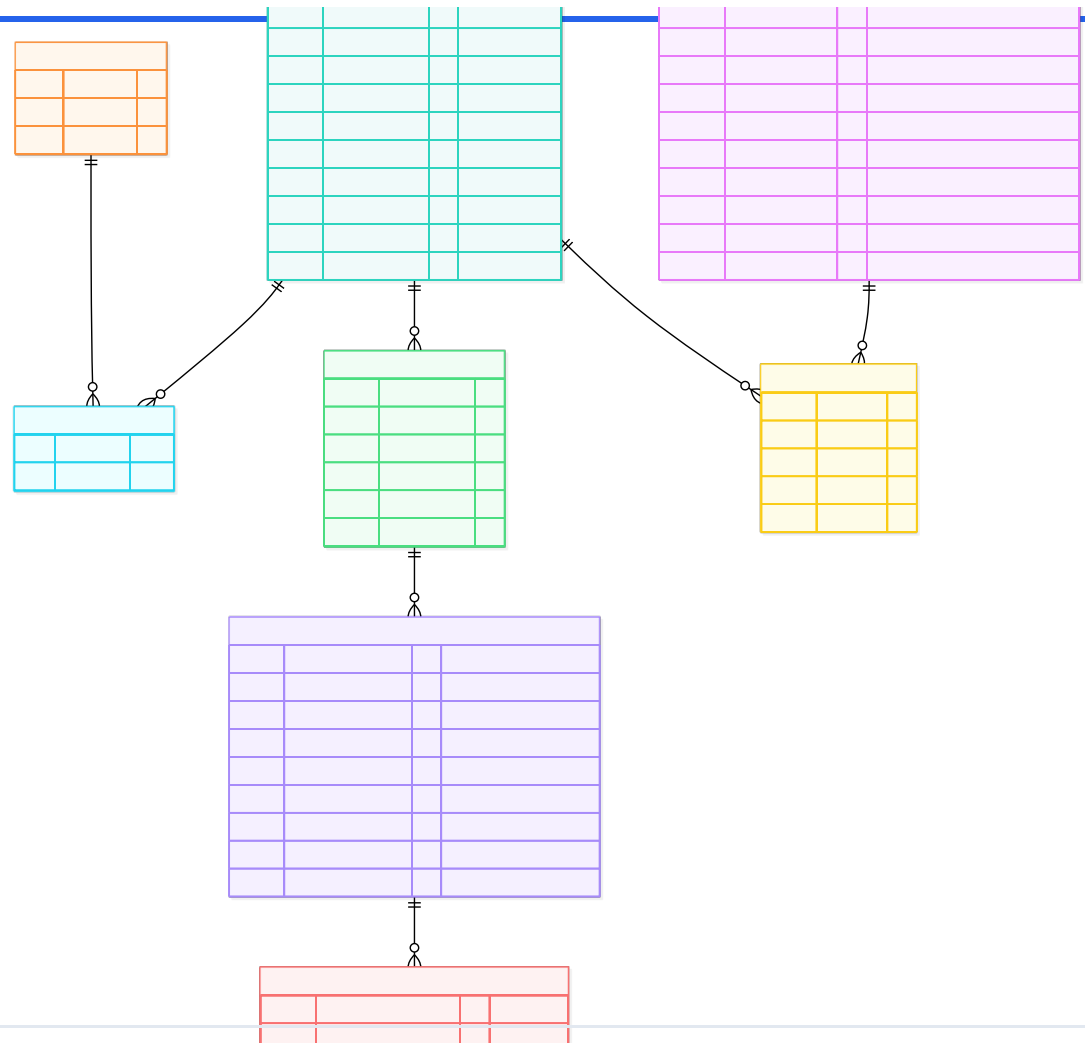




## 8. Thiết kế CSDL & Polyglot Persistence

### Kiến trúc lưu trữ đa tầng (Polyglot):

- **PostgreSQL:** Lưu dữ liệu nghiệp vụ quan hệ (Users, Comics, Chapters), quản lý hạn mức AI.
- **Elasticsearch:** Tìm kiếm mờ tên truyện đa ngôn ngữ bằng chỉ mục đảo ngược.
- **Redis & Caffeine:** Cache phân tán/cục bộ tăng tốc phản hồi API.
- **MinIO:** Object Storage lưu ảnh sách và JSON metadata.



## 9. Thiết kế siêu dữ liệu (Metadata)

---

**Ý tưởng cốt lõi:** Không vẽ đè chữ lên ảnh. Lưu trữ metadata JSON riêng biệt.

### **original\_metadata.json (Tọa độ gốc):**

- Lưu danh sách `bubbles[]`.box: Tọa độ pixel  $[x, y, w, h]$ .
- Lưu chữ gốc `original_text` và phiên âm romanization.

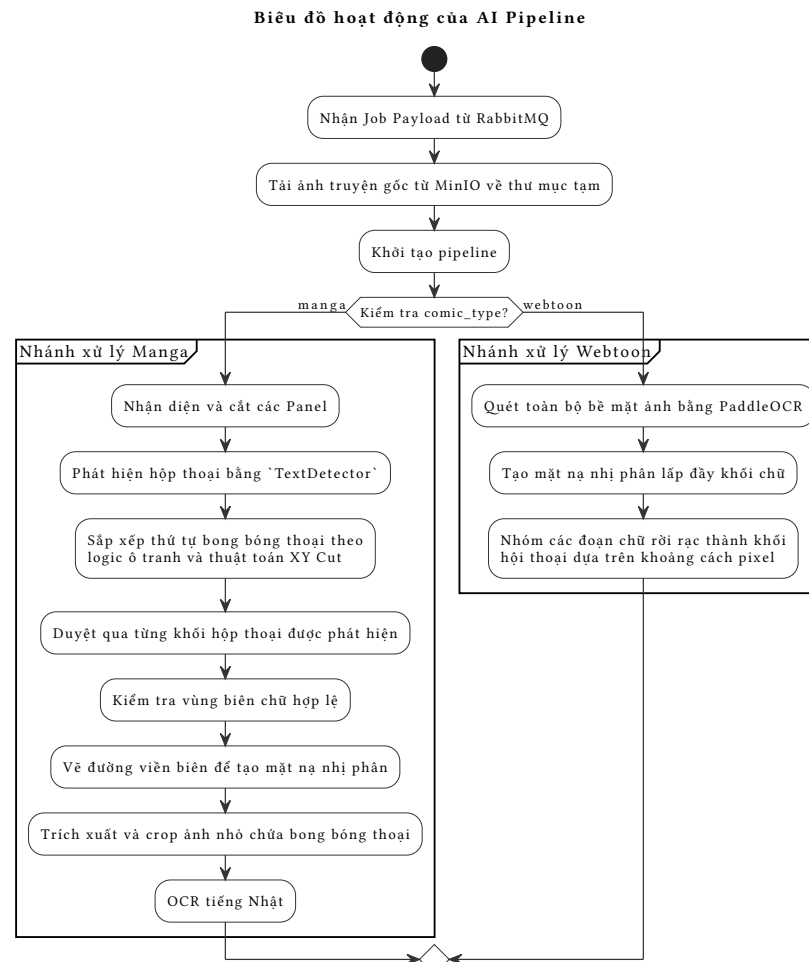
### **translation\_metadata.json (Bản dịch):**

- Ánh xạ qua `bubbles[]`.id trùng khớp với tệp gốc.
- Lưu dịch nghĩa `full_translation` và chi tiết giải nghĩa từng từ `chunk_meanings` (cho pop-up).

## 10. Luồng xử lý tại AI Pipeline

### Các bước xử lý chính:

- **OCR:** YOLOv12x (phân tách Panel) + Comic Text Detector (định vị hộp thoại) + MangaOcr/PaddleOCR.
- **Inpainting:** Tẩy chữ gốc và phục hồi nền bằng LaMa (FFC).
- **Dịch thuật:** Gemini 3.1 Flash-Lite dịch đa phương thức với ảnh **Set-of-Mark (SoM)**.
- **Lưu trữ:** Đẩy ảnh sạch + JSON lên MinIO và gọi Webhook.



## 11. Cài đặt và Triển khai thực tế

---

### Frontend (Next.js 16 & React 19):

- **Zustand:** Quản lý state phiên làm việc.
- **React Query & Axios:** Caching & API sync.
- **Tailwind CSS v4:** Responsive & Dark Mode.
- **React Intl:** Bản địa hóa đa ngôn ngữ.

### Backend & AI Service:

- **Spring Boot 3.5:** Bảo mật JWT, JPA, Elasticsearch, Redis.
- **FastAPI:** Lập trình bất đồng bộ, google-genai SDK, aio-pika, ultralytics, simple-lama, paddleocr.
- **Docker & Compose:** Đóng gói container đồng bộ toàn hệ thống.

## 12. Kiểm thử & Đảm bảo chất lượng

---

### Kiểm thử Frontend & Backend:

- **Frontend:** Phân tích tĩnh với ESLint & TypeScript.
- **Backend:**
  - JUnit 5 & Mockito: Test đơn vị và tích hợp.
  - JaCoCo: Đo lường độ bao phủ code coverage.
  - PiTest (Mutation Testing): Đánh giá chất lượng test case qua lỗi đột biến nhân tạo.

### Kiểm thử AI Pipeline:

- **pytest:** Test đơn vị thuật toán sắp xếp ô tranh (panel\_sorter, XY Cut).
- **test\_pipeline.py:** CLI tool test tích hợp cục bộ độc lập (không cần RabbitMQ).
- **Visual/Position Debugger:** Xuất ảnh bounding box trung gian kiểm tra trực quan.

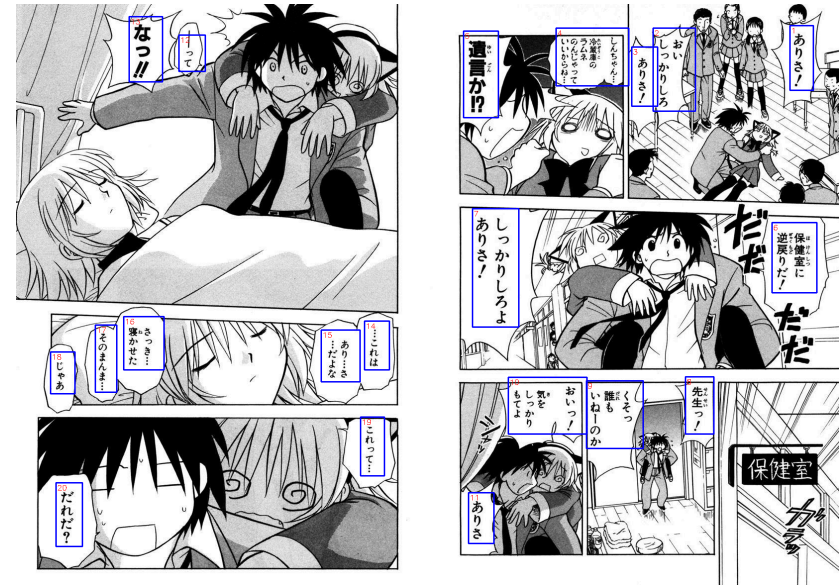
## 13. Đánh giá chất lượng & Demo

### Đo lường định lượng:

- Thời gian xử lý: 11.17s/trang (gồm dịch Gemini API 1.5-3s, còn lại OCR/Inpaint CPU cục bộ).
- Tỷ lệ lỗi ký tự (CER): MangaOcr 6-10%, PaddleOCR 5-8%.

### Đánh giá định tính:

- LaMa Inpainting: Tái tạo nền sạch, giữ nguyên screentones Manga.
- Gemini 3.1 Flash-Lite: Dịch tự nhiên, xưng hô chuẩn xác nhờ ngữ cảnh hình ảnh.



Ảnh gỡ lỗi nhận diện hộp thoại (Arisa © Yagami Ken)

## 14. Kết luận & Hướng phát triển

---

### Kết quả đạt được:

- Xây dựng thành công **AI Pipeline** tự động hóa dịch thuật truyện tranh.
- Áp dụng **RabbitMQ** xử lý bất đồng bộ, giải quyết nút thắt cổ chai.
- **Tách biệt** lớp phủ chữ động và ảnh sạch, tối ưu lưu trữ.

### Định hướng tương lai:

- Nghiên cứu **fine-tune OCR** nội bộ cho font chữ nghệ thuật/viết tay.
- Nâng cấp inpainting với **GAN/Diffusion** cải thiện mỹ thuật bối cảnh.
- Triển khai **Autoscaling** AI Worker trên Cloud theo hàng đợi RabbitMQ.